

One Person, One ID, Everywhere

How MissionIQ, Family Graph, and Doc Anonymizer share a lifelong identifier for every family and every person.

Every parishioner, parent, and student now carries one permanent ID (I... for a person, F... for a family) that no product ever issues twice, and a roster can be sent through AI without a single real name leaving the school's network.

1,289

people imported from MissionIQ demo data in 4 seconds

10

questions a person had to answer; the rest settled automatically

0

wrong matches on a 500-family test roster

1,055

automated tests passing across both products

SITUATION

School rosters and parishioner lists are about to run through Doc Anonymizer so AI can work on them safely. Without a shared identifier, the same mother would get a different code in every file, and nobody could tell that "Maria Garcia" on the parish list and on the school roster is one person. MissionIQ already holds most of the community, some of it linked by a looser matcher.

WHAT WE DID

- Made Family Graph the only system that issues IDs, and gave its codes a readable form: `I` + 16 hex for a person, `F` + 16 hex for a family.
- Built a one-time, read-only MissionIQ import that links every MissionIQ record to its Family Graph person, so later lookups are exact, not guesses.
- Connected Doc Anonymizer to Family Graph for spreadsheets: names become IDs, questions appear in the preview, and nothing is written until you confirm.
- Rewrote matching so obvious cases settle on their own in both directions, and only real ambiguity reaches a person.
- Locked it down: everything stays inside the firewall, every data call needs a scoped key, and logs hold counts, never names.
- Ran an independent bug hunt: 40 defects fixed, plus 10 caught reviewing the fixes.

RESULTS

- **10 of 1,289** people needed a decision on the MissionIQ import; a re-run needed **0**.
- **0 wrong matches** and **30 questions** (1 in 33 people) on a 500-family roster; before the rework it was 884 questions.
- Fixed old matcher bugs that could merge spouses or neighbors.

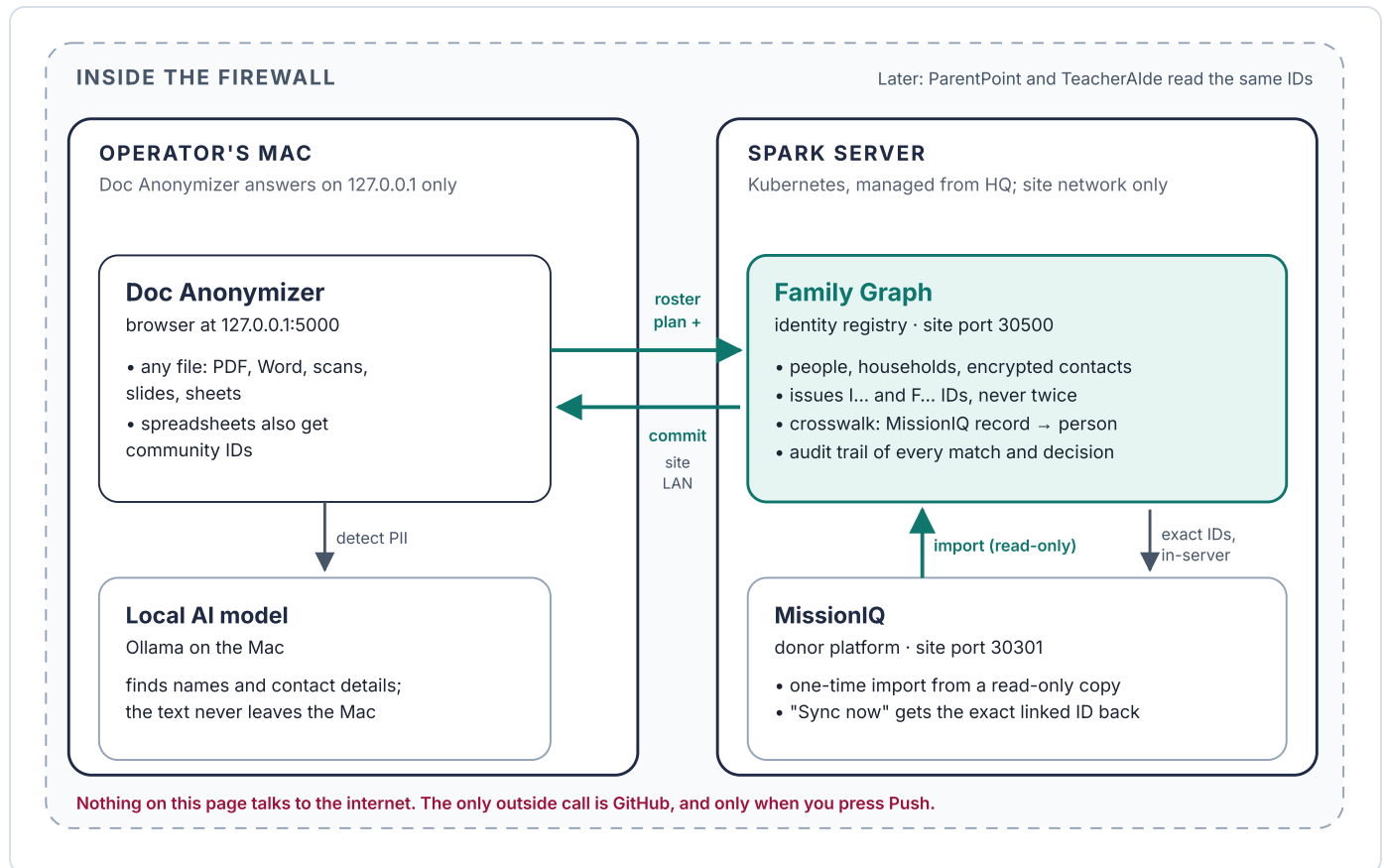
WHAT'S NEXT

- Upgrade Family Graph on the Spark server, then run `import-missioniq` once, answering its few questions.
- Connect MissionIQ with a key named `missioniq` and press Sync now once.
- Anonymize the first real roster and keep the key file in the backup routine.

THE SYSTEM MAP

Three products, one identity registry, one firewall

Family Graph is the registry: it alone decides whether someone already exists and issues IDs. MissionIQ and Doc Anonymizer ask it; they never mint an identity of their own.



WHO OWNS WHAT

- Family Graph** owns identity: who exists, who belongs to which household, and every ID.
- MissionIQ** owns giving and donor work. It keys that work to Family Graph IDs.
- Doc Anonymizer** owns documents. It asks Family Graph for IDs and keeps the key that restores names.

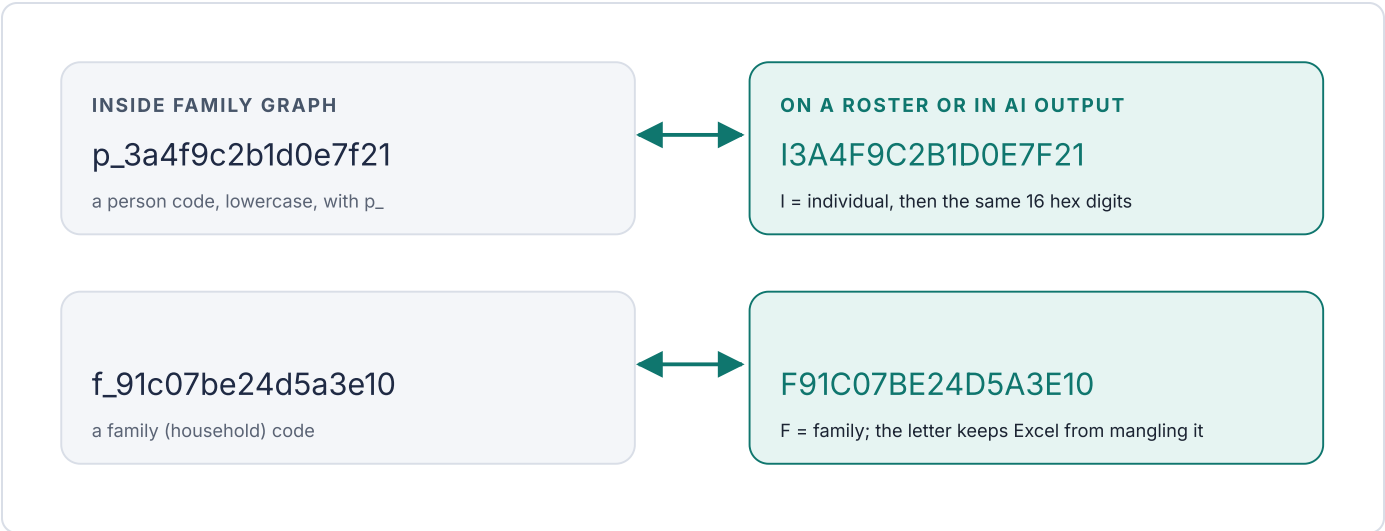
WHAT EACH SIDE CAN SEE

- The AI model sees only anonymized text, never a real name.
- Doc Anonymizer's key carries one scope, `roster : plan` and `commit` rosters, look up an ID. It cannot browse the directory, change settings, or delete anyone.
- MissionIQ's file is opened read-only; nothing is ever written back into it.

THE IDENTIFIER

One code, two spellings, never reissued

The community ID is Family Graph's own code, written so a spreadsheet and a person can read it. Nothing is translated or stored twice.



THE PROMISES

- **One human, one ID, for life.** The same mother keeps her ID on the parish list, next year's roster, and in MissionIQ.
- **Never reissued.** An ID is never given to anyone else, even after the person leaves.
- **Merges redirect.** If two records turn out to be one person, the old ID points to the survivor forever, so older files still restore.
- **Nothing minted on a guess.** An ID is created only after every open question is answered and you confirm.
- **Retry-safe.** If the connection drops mid-save, resending can never create a second ID; the file stays locked until the outcome is known.

THE CROSSWALK

- Each imported MissionIQ record is linked to its person: contact:<id>, child:<id>, family:<id>.
- From then on, "MissionIQ contact X" *is* that person. Lookups are exact, never re-matched by name.
- Only a deliberate import writes a link. Everyday lookups read links; they never create them, so a loose guess can never become permanent.
- If a linked record later changes into someone else (a different first name and birthdate), it comes back as a question.

WHERE THE ID SHOWS UP IN A SCRUBBED SPREADSHEET

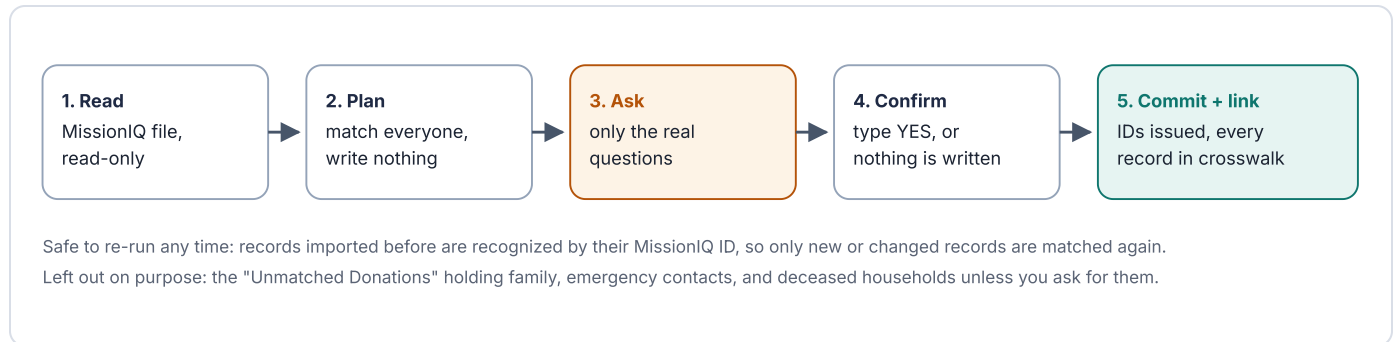
COLUMN	BEFORE	AFTER
Student First Name	Emma	[I3A4F9C2B1D0E7F21]
Student Last Name	Smith	[PERSON_7C1B0A94E2D3] (per-value token, as before)
Parent Email	jane@example.org	[EMAIL_3F0A91C2B7E4]
Household (added if missing)		[F91C07BE24D5A3E10] , removed again on restore

PDFs, Word files, scans, and slides are anonymized exactly as before. Community IDs apply only to spreadsheets, and only when Family Graph is connected.

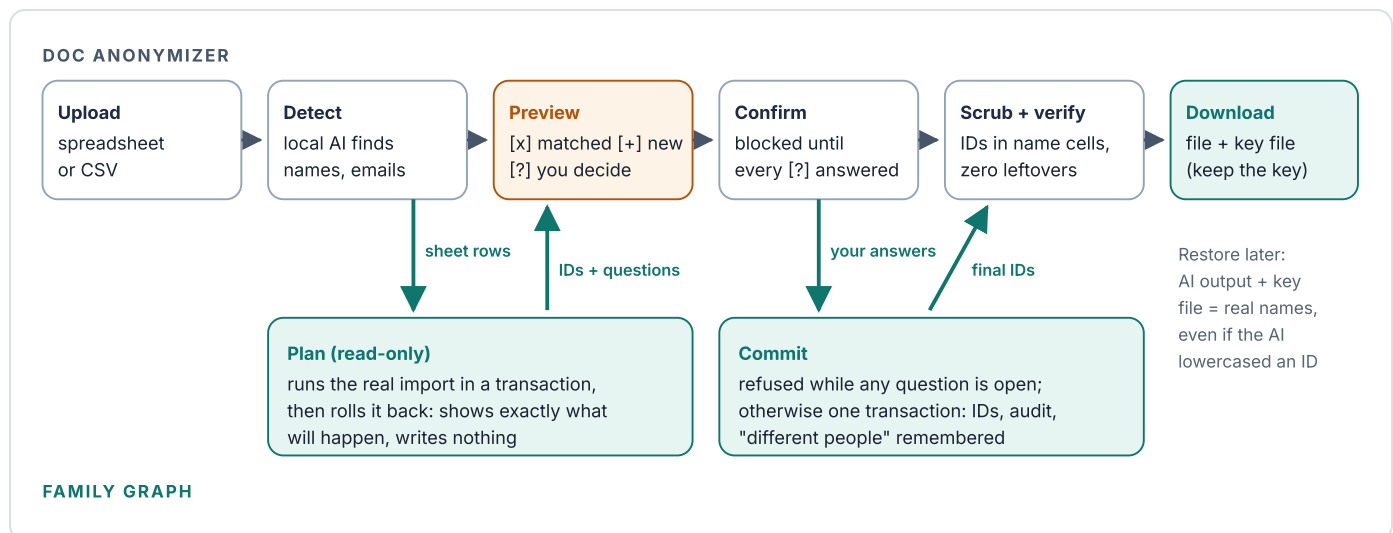
THE TWO FLOWS

Bringing people in, and sending a roster through AI

FLOW A · ONE-TIME MISSIONIQ IMPORT (COMMAND LINE ON THE SPARK SERVER)



FLOW B · A ROSTER THROUGH DOC ANONYMIZER (BROWSER)



If Family Graph is unreachable, Doc Anonymizer says so and stops the community step; it never quietly falls back to a file without IDs.

WHAT YOU WILL SEE

```
MissionIQ: 300 households, 667 adults,  
          622 students  
people:   1279 new, 10 need you  
households: 300 new, 0 need you  
  
[4/10] PERSON Rosa Schmidt grandparent  
       in Schmidt Family  
why: a possible match, not a certain one  
1) same person as I5E0B2C7A91D4F386  
   Rosa Schmidt adult Schmidt Family  
n) a different person - give them a new id  
s) not a real person - leave them out  
q) stop now - nothing is written  
> 1
```

The MissionIQ import in Terminal. A number means "same person", n means new. Nothing is written until you type YES at the end.

```
COMMUNITY IDS - who is who, from Family Graph  
PEOPLE      412 known  37 new  2 need you  
HOUSEHOLDS  160 known  12 new  0 need you  
            9 told apart automatically  
  
[?] ROW 14 PERSON Emma Lee (child)  
possible match, not a certain one  
Emma Lee I5E0B...F386  
child - grade 3 - Lee Family  
[ SAME PERSON ]  
[ NEW PERSON ] [ NOT A PERSON ]  
  
[ CONFIRM AND SCRUB ] stays off until  
every [?] is answered
```

The Doc Anonymizer preview. Every other row shows [x] known or [+] new, with a way to overrule it.

HOW MATCHING DECIDES

Automatic when it is obvious, a question only when it is not

Each incoming person is compared with everyone Family Graph knows. Three outcomes:

SAME PERSON, AUTOMATICALLY

- Same email or phone *and* the first name lines up (Lupe = Guadalupe, Bob = Robert)
- Same first and last name *and* same birthdate
- Same first and last name *and* same home address
- A student listed by name only, in a household we already know
- A MissionIQ record imported before (the crosswalk)

DIFFERENT PERSON, AUTOMATICALLY

- Shared inbox, different first name: a spouse or a child, not the same person
- Jr and Sr, or II and III
- Different birthdates, with nothing else shared
- An adult row against a child record at one address (John and John Jr.)
- In one upload: same-named students in different grades, or same-named parents with different emails and homes

ASK A PERSON

- Same first and last name, nothing to confirm or rule out
- Same name, same inbox, different birthdate (maybe a typo)
- Two possible matches at once
- A stored MissionIQ link that no longer fits the record
- The same name twice in one household
- Look-alike first names (Luis / Luisa)
- Placeholders ("TBD"), initials, or a business name

HOUSEHOLDS FOLLOW THE SAME IDEA

EVIDENCE	OUTCOME	WHY
People already known on the row all belong to one household	same	They vouch for it, in the role they hold there.
Someone new shares the family email or phone, and a surname fits	same	A spouse on the family inbox joins the family.
Same exact address and a surname in common	same	An address alone is not enough; people move.
Same address, no surname in common, nobody known	new	A different family moved in.
A grown-up child now listed as a parent, or a grandparent on a daughter's row	new	Their other household does not pull this one in.
Only children are known and a new adult lives elsewhere	ask	Could be the other parent's home (divorced parents).

MEASURED ON TEST DATA

RUN	PEOPLE	QUESTIONS	WRONG MATCHES
MissionIQ demo data, first import	1,289	10	0
Same import, run again	1,289	0	0
500-family roster, first time (before the rework)	1,000	884	0
500-family roster, first time (after)	1,000	30	0
Same roster, re-imported	1,000	0	0

Security, logging, and the commands you will use

SECURITY POSTURE

- Family Graph and MissionIQ run in Kubernetes on the Spark server, managed from HQ, and answer only on the site network. MissionIQ reaches Family Graph inside the server.
- Every Family Graph data call needs a key. Port 30500 is plain HTTP, so keep it on a staff-only network; guest or student Wi-Fi must not reach it.
- Doc Anonymizer runs on your Mac, answers only on 127.0.0.1, and refuses any Family Graph address outside the local network. No override.
- Doc Anonymizer's Family Graph key carries only the `roster` scope. It is stored in a file only the owner can read, never shown again, never logged.
- Contact details in Family Graph are encrypted at rest; searches use keyed hashes.
- Uploaded files are deleted the moment processing ends or fails.

WHAT THE LOGS HOLD

- **Logged:** file name, sizes, counts matched / new / asked, import run code, timings, and full error details.
- **Never logged:** names, emails, phones, addresses, document text, replacement values, or keys.
- Family Graph: `server.log` and `cli.log` under `/data/logs` in its pod.
- MissionIQ: `/data/logs/familygraph.log` in its pod.
- Doc Anonymizer: `~/doc-anonymizer/anonymizer.log` on the Mac.
- Paste the last lines into a chat when something breaks; they are safe to share.

FIRST-TIME SETUP ON THE SPARK SERVER

```
# HQ: build Family Graph 5a698e6, merge the edge-ops branch, sync familygraph
# On the Spark server, as root:
FG="k3s kubectl -n familygraph exec -it deploy/familygraph --"
$FG node bin/family-graph.js issue-key missioniq           # token goes into Vault
$FG node bin/family-graph.js issue-key docanonymizer roster # token goes into Doc Anonymizer
# copy MissionIQ's database into the pod (runbook step 4), then:
$FG node bin/family-graph.js import-missioniq /tmp/missioniq.db --dry-run
$FG node bin/family-graph.js import-missioniq /tmp/missioniq.db --category church
# HQ: sync missioniq-secrets, then missioniq. In MissionIQ press Sync now once.
```

Every command, in order, with checks and rollback: `familygraph-missioniq-connect.md` in the edge-ops repo. In Doc Anonymizer's [FAMILY GRAPH] panel use `http://[box-ip]:30500` and the docanonymizer key. The missioniq key goes into Vault, never into a chat or a file. Import before the first Sync now, so every MissionIQ record links by its own id. The dry run shows the counts and questions; the real run writes only after you type YES.

KEEP IN THE BACKUP ROUTINE

- Family Graph: `family-graph backup <passphrase>` in its pod (encrypted, lands on its volume; copy it off the box).
- Doc Anonymizer: the `keys/` folder on the Mac. Without a key file an anonymized document cannot be restored.

KNOWN FOLLOW-UPS

- MissionIQ: "re-check all" button, student sync, and household re-stamp, so new contacts use exact links. Until then, re-run the import to link new records.
- Call Center: the goal is to run it on the local network too. Today it is a Firebase app, not set up on the server; the rebuild has not started.
- Doc Anonymizer is not on the server yet; moving it there would keep roster traffic off the LAN.